

▼ Glioma: Segmentação de Sub-regiões + Graduação (SAM 2 / MedSAM + Atenção + LoRA)

Projeto final — Visão Computacional Avançada · FCTE/UnB

Pipeline de duas etapas *acopladas* que **(A) segmenta** as sub-regiões do tumor (necrose/NCR, edema/ED, tumor ativo/ET) e **(B) gradua** a lesão (LGG × HGG) a partir da máscara segmentada. A graduação depende explicitamente da segmentação — **não** é um classificador de imagem isolado.

Introdução

Contexto clínico

A graduação histológica de gliomas (baixo grau/LGG × alto grau/HGG, classificação OMS) orienta diretamente a conduta clínica — de vigilância a ressecção agressiva seguida de radioquimioterapia. Hoje ela depende de biópsia ou de leitura radiológica manual da MRI, ambas sujeitas a variabilidade inter-observador e a atraso no laudo (Louis et al., 2021). As sub-regiões do tumor (necrose/NCR, edema peritumoral/ED, tumor realçante/ET) já orientam informalmente a impressão do radiologista sobre o grau — o que motiva **automatizar essa cadeia**: segmentar as sub-regiões e, a partir delas, inferir o grau.

Problema

Duas limitações recorrentes na literatura motivam o desenho deste trabalho:

1. **Segmentação e graduação tratadas como tarefas independentes.** A maioria dos pipelines treina um classificador de imagem "cru" para o grau, sem restringir a decisão à região tumoral — o que reduz a interpretabilidade e ignora um sinal forte (a geometria do tumor segmentado).
2. **Fine-tuning completo de encoders de fundação é caro.** Adaptar um encoder pré-treinado em bilhões de máscaras (SAM 2/MedSAM) ao domínio médico tipicamente exige atualizar todos os pesos, o que não cabe na memória de uma GPU T4 (16 GB) nem é necessário dado o tamanho pequeno dos datasets médicos (centenas, não milhões, de casos).

Objetivos

- Segmentar as sub-regiões do glioma (NCR/NET, ED, ET) com um decoder de atenção sobre um encoder de fundação **congelado**, adaptado por **LoRA** (poucos % dos parâmetros treináveis).
- **Acoplar** a graduação (LGG × HGG) à máscara segmentada via *masked-attention pooling*, em vez de um classificador de imagem isolado.
- Reportar as métricas oficiais do desafio BraTS (Dice/IoU/Sens/Spec/HD95), além de AUC/calibração da graduação, uma **ablação** que isola o ganho do encoder de fundação, **biomarcadores** interpretáveis, **incerteza** (MC-Dropout/TTA) e **explicabilidade** — entregáveis científicos além do mínimo do desafio.

Resumo (o que este notebook entrega)

#	Entregável	Seção
1	Segmentação multi-classe com Attention U-Net sobre encoder de fundação (SAM 2/MedSAM) adaptado por LoRA	3, 6, 7
2	Graduação guiada pela máscara (masked-attention pooling + descritores geométricos)	7
3	Métricas do desafio BraTS: Dice / IoU / Sens / Spec / HD95 por sub-região (WT/TC/ET)	7
4	Graduação: AUC/ROC , matriz de confusão, F1, e calibração (ECE, Brier, reliability)	7
5	Ablação científica : encoder de fundação (LoRA) × congelado × U-Net do zero	8
6	Biomarcadores tumorais derivados da segmentação → ponte interpretável seg→grau	9
7	Incerteza (MC-Dropout + TTA): mapas de "onde o modelo hesita"	10
8	Explicabilidade da graduação (atenção intrínseca + saliência por oclusão)	11

Pronto para a GPU T4 do Colab. O notebook roda ponta-a-ponta em dados **sintéticos** (sem download) para validar tudo; troque uma flag para treinar no **BraTS real**.

Como usar

1. Runtime → Change runtime type → T4 GPU.
2. Rode as células **1–3** (setup + sanity-check sintético).
3. Para dados reais: rode a **Seção 4** (download BraTS2020 via Kaggle) e ajuste `DATA_SOURCE = "brats"` na Seção 5.

4. Rode 5→12 para treino, avaliação e todas as análises.

As referências completas citadas ao longo do notebook estão consolidadas na Seção 14 (Referências), ao final.

1. Ambiente, GPU e dependências

```
# --- No Colab: clona o repositório (idempotente -- pula se já clonado) ---
# IMPORTANTE: "!comando && %magic" NAO funciona -- '%cd' e' magic do IPython,
# nao comando de shell; misturado com '!' e '&&' o bash tenta executa-lo e
# falha (erro "fg: no job control") sem trocar de pasta de verdade. Por isso
# o clone e o %cd vao em linhas separadas abaixo.
REPO_ROOT = '/content/glioma_seg_grade' # ajuste se clonar em outro lugar
![-d glioma_seg_grade ] || git clone https://github.com/Bappoz/glioma-seg-grad glioma_seg_grade
%cd glioma_seg_grade
!pip -q install -r requirements.txt

import os, sys
# path ABSOLUTO (nao '.') -- continua valido mesmo que o cwd mude depois, e
# permite que outras celulas (ex.: download do Kaggle) re-afirmem o path sem
# depender desta celula ter rodado antes na MESMA sessao do kernel.
if REPO_ROOT not in sys.path:
    sys.path.insert(0, REPO_ROOT)

import torch, platform
print('Python :', platform.python_version())
print('PyTorch:', torch.__version__)
print('CUDA  :', torch.cuda.is_available())
if torch.cuda.is_available():
    print('GPU  :', torch.cuda.get_device_name(0),
          f"({torch.cuda.get_device_properties(0).total_memory/1e9:1f} GB)")
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
```

```
Cloning into 'glioma_seg_grade'...
remote: Enumerating objects: 185, done.
remote: Counting objects: 100% (185/185), done.
remote: Compressing objects: 100% (83/83), done.
remote: Total 185 (delta 91), reused 177 (delta 83), pack-reused 0 (from 0)
Receiving objects: 100% (185/185), 126.30 KiB | 2.48 MiB/s, done.
Resolving deltas: 100% (91/91), done.
/content/glioma_seg_grade/glioma_seg_grade
Python : 3.12.13
PyTorch: 2.11.0+cu128
CUDA   : True
GPU    : NVIDIA A100-SXM4-40GB (42.4 GB)
```

```
# Reprodutibilidade
import numpy as np, random
def set_seed(seed=42):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
set_seed(42)

# matplotlib inline + estilo limpo
import matplotlib.pyplot as plt
plt.rcParams.update({'figure.dpi': 110, 'font.size': 11, 'axes.grid': True, 'grid.alpha': 0.3})
```

2. Metodologia

Arquitetura em dois estágios acoplados

Estágio A — Segmentação. Um encoder de fundação (SAM 2 / MedSAM 2, backbone *Hiera*; Ravi et al., 2024; Ma et al., 2024; Ryali et al., 2023) — **congelado** — extrai features multi-escala. Um decoder com **Attention Gates** (Oktay et al., 2018), inspirado na U-Net original (Ronneberger et al., 2015), funde essas escalas por *skip-connections* e reconstrói a máscara multi-classe {fundo, NCR/NET, ED, ET}.

Estágio B — Graduação. As features do encoder são agregadas por um **masked-attention pooling** guiado pela máscara predita no Estágio A e concatenadas a **descritores geométricos** (frações de área por sub-região). A saída é o grau (LGG × HGG). A decisão só "olha" para o tumor segmentado → decisão interpretável e alinhada à clínica.

Por que SAM 2 + atenção supera uma U-Net pura?

- O encoder Hiera vem **pré-treinado em >1 bilhão de máscaras**: traz priors de bordas/objetividade que uma U-Net do zero (Ronneberger et al., 2015) em poucas centenas de volumes não aprende — quantificado na ablação da Seção 8.
- **Attention Gates** suprimem crânio/edema difuso → bordas mais nítidas.

- O **acoplamento seg→grau** torna a graduação dependente da região segmentada.

Comparação controlada: fundação real (SAM 2) × encoder aleatório (stub)

Para **isolar o ganho atribuível ao pré-treinamento** (e não à arquitetura), o notebook roda a MESMA pipeline em dois modos, alternados por uma única flag `BACKBONE` (Seção 5): `sam2` usa o encoder Hiera pré-treinado em **>1 bilhão de máscaras**; `stub` usa um encoder de **mesma arquitetura, porém com pesos aleatórios**. Todos os demais fixes metodológicos e hiperparâmetros são idênticos nos dois modos — só o encoder muda —, então qualquer diferença nos resultados finais mede o efeito do pré-treinamento. Cada modo salva seus artefatos separadamente (sufixo `_stub` / `_sam2`) e os dois são reportados lado a lado nas Seções 8 e 13.

Fine-tuning eficiente com LoRA

O encoder fica congelado; injetamos adaptadores de baixo posto **LoRA** (Hu et al., 2021) nas projeções de atenção. Treinam ~1–5% dos pesos → **cabe na T4 e preserva os priors** (evita *catastrophic forgetting*). `lora_B=0` no início ⇒ o modelo parte exatamente do encoder pré-treinado; `merge_all_lora()` funde para inferência sem custo.

Perdas e métricas

- Segmentação: **Dice + CE** (padrão-ouro) ou **Focal-Tversky** (Abraham & Khan, 2019). — `β>α` prioriza *recall* no ET, a sub-região menor e mais difícil.
- Graduação: **CE com label smoothing** (Müller et al., 2019) — reduz overconfidence e melhora a calibração.
- Métricas: Dice/loU/Sens/Spec/HD95 (seg) · Acc/AUC/F1/ECE (Guo et al., 2017) /Brier (Brier, 1950) (grau).

Otimização — por que estes hiperparâmetros

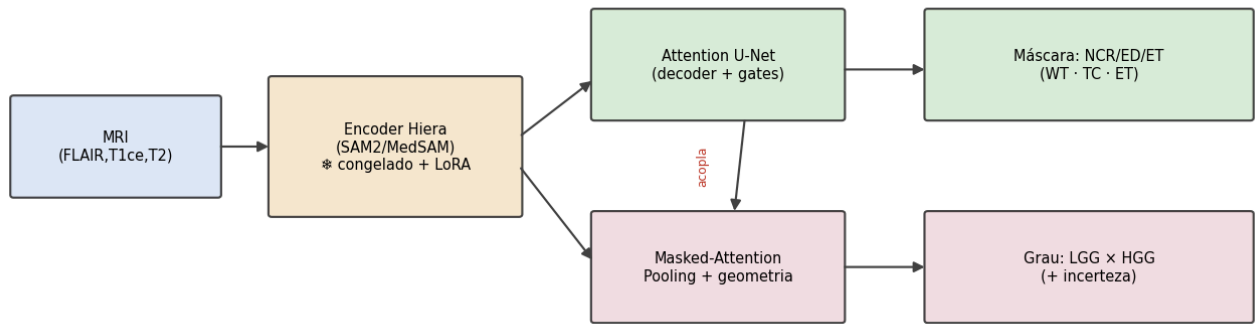
- **AdamW** (Loshchilov & Hutter, 2019) com `lr=3e-4`, `weight_decay=1e-4`: desacopla o *weight decay* do momento (ao contrário do Adam clássico), o que importa aqui porque só uma fração pequena dos parâmetros (LoRA + decoder) é treinada e não pode ser regularizada de menos nem de mais.
- **Warmup linear (3 épocas) + cosine annealing** até `min_lr_ratio=0.02` (Loshchilov & Hutter, 2017 — *SGDR*): o warmup evita que os adaptadores LoRA, inicializados com `B=0` (saída nula), recebam um passo grande antes de "aprender a direção" certa; o cosine evita o platô abrupto de um *step decay* e foi o que motivou treinar por **30 épocas** em vez de 10 (o *schedule* zerava o `lr` cedo demais com menos épocas, subtreinando).
- **Gradient clipping** (`grad_clip=1.0`) + **AMP** (mixed precision): a T4 só cabe o treino em `float16`; o clipping estabiliza os picos de gradiente típicos de precisão mista.
- **Acumulação de gradiente** (`accum_steps=2`): simula um batch efetivo maior sem aumentar o pico de memória — necessário com `img_size=256` na T4.
- **Peso de classe inverso-frequência na CE de graduação** (`auto_grade_weight=True`): o BraTS é dominado por casos HGG; sem o peso, o modelo colapsa para sempre prever HGG (alta *accuracy*, AUC ruim).
- `p_drop=0.1` fica **ativo mesmo fora do treino** de propósito — não é só regularização: é o que habilita o **MC-Dropout** da Seção 10 (com dropout zerado, a incerteza epistêmica colapsaria a zero).

```
# Diagrama da arquitetura (renderizado com matplotlib — sem dependências externas)
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

def draw_architecture():
    fig, ax = plt.subplots(figsize=(12, 4.2)); ax.axis('off')
    ax.set_xlim(0, 12); ax.set_ylim(0, 4.2)
    def box(x, y, w, h, text, color):
        ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle='round,pad=0.03',
            linewidth=1.4, edgecolor='#333', facecolor=color, alpha=0.9))
        ax.text(x + w/2, y + h/2, text, ha='center', va='center', fontsize=9.5)
    def arrow(x1, y1, x2, y2):
        ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle='->',
            mutation_scale=14, linewidth=1.4, color='#444'))
    box(0.2, 1.6, 1.9, 1.0, 'MRI\n(FLAIR,T1ce,T2)', '#DCE6F5')
    box(2.6, 1.4, 2.3, 1.4, 'Encoder Hiera\n(SAM2/MedSAM)\n* congelado + LoRA', '#F5E6CC')
    box(5.6, 2.4, 2.3, 1.1, 'Attention U-Net\n(decoder + gates)', '#D8EAD3')
    box(5.6, 0.3, 2.3, 1.1, 'Masked-Attention\nPooling + geometria', '#F3D9DE')
    box(8.7, 2.4, 3.0, 1.1, 'Máscara: NCR/ED/ET\n(WT · TC · ET)', '#D8EAD3')
    box(8.7, 0.3, 3.0, 1.1, 'Grau: LGG × HGG\n(+ incerteza)', '#F3D9DE')
    arrow(2.1, 2.1, 2.6, 2.1)
    arrow(4.9, 2.2, 5.6, 2.8); arrow(4.9, 1.9, 5.6, 0.9)
    arrow(7.9, 2.9, 8.7, 2.9); arrow(7.9, 0.85, 8.7, 0.85)
    arrow(7.0, 2.4, 6.9, 1.4) # máscara guia a graduação (acoplamento)
    ax.text(6.55, 1.9, 'acopla', fontsize=8, color='#C0392B', rotation=90, va='center')
    ax.set_title('Pipeline: segmentação (topo) → graduação guiada pela máscara (base)', fontsize=11)
    plt.tight_layout(); return fig
```

```
draw_architecture(); plt.show()
```

Pipeline: segmentação (topo) → gradação guiada pela máscara (base)



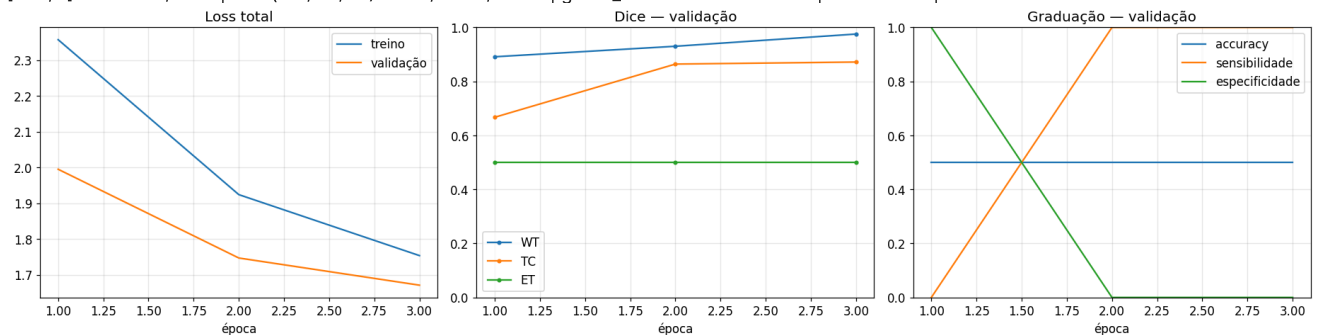
3. Sanity-check com dados sintéticos (valida o pipeline sem download)

`SyntheticBraTS` gera anéis aninhados (edema → core → ET) como "tumor"; ~40% dos casos são LGG-like (sem ET), dividindo o rótulo de grau em 2 classes para a curva ROC funcionar. Serve para confirmar shapes e o loop antes do BraTS real.

```
from src.train import Trainer, TrainConfig
from src.dataset import SyntheticBraTS, SegAugmentation
from torch.utils.data import DataLoader
from src import viz

sanity_cfg = TrainConfig(backbone='stub', epochs=3, batch_size=16, img_size=128,
                        region='tversky', p_drop=0.1, warmup_epochs=1,
                        out_dir='runs/sanity')
tr = DataLoader(SyntheticBraTS(96, img_size=128, transform=SegAugmentation()),
                batch_size=16, shuffle=True)
va = DataLoader(SyntheticBraTS(32, img_size=128), batch_size=16)
sanity_hist = Trainer(sanity_cfg).fit(tr, va)
viz.plot_training_curves(sanity_hist); plt.show()
print('Pipeline OK — shapes e loop validados.')
```

[001/3] loss 2.357/1.995 | Dice(WT/TC/ET) 0.891/0.667/0.500 | grade_acc 0.500 AUC 0.791 | lr 3.00e-04 | 1s
 [002/3] loss 1.924/1.747 | Dice(WT/TC/ET) 0.930/0.864/0.500 | grade_acc 0.500 AUC 0.420 | lr 1.53e-04 | 0s
 [003/3] loss 1.753/1.671 | Dice(WT/TC/ET) 0.975/0.872/0.500 | grade_acc 0.500 AUC 0.486 | lr 6.00e-06 | 1s



4. Aquisição do BraTS real

Pule esta seção se quiser apenas a demonstração sintética. Para treinar de verdade, escolha **uma** das opções abaixo. A **Opção A (Kaggle BraTS2020)** é a mais simples e é a que este notebook cabeia diretamente.

O dataset de referência é o **BraTS — Multimodal Brain Tumor Segmentation Challenge** (Menze et al., 2015; Bakas et al., 2017; Bakas et al., 2018): 4 modalidades de MRI por paciente (T1, T1ce, T2, FLAIR), com máscaras de sub-região (NCR/NET, ED, ET) anotadas por especialistas e, nas edições com metadado de grau, o rótulo histológico OMS por paciente.

Opção A — Kaggle · BraTS2020 (recomendada)

Mirror estável (`awsaf49/brats20-dataset-training-validation`, 369 casos) com a estrutura oficial `BraTS20_Training_XXX/*_flair.nii ...` e o grau real em `name_mapping.csv` (coluna `Grade` = HGG/LGG). Requer um token do Kaggle: **Kaggle → Account → Create New API Token** → baixa `kaggle.json` → suba para o Colab.

Procedência: mirror de terceiros (não o portal oficial do desafio) — declare isso no relatório.

```
# --- Opção A: download do BraTS2020 via API do Kaggle ---
# Guarda defensiva: se esta célula for executada isoladamente (ex.: depois de
# reiniciar o runtime e pular direto para cá, sem re-rodar a Seção 1), garante
# que 'src' continue importável mesmo sem ter rodado a célula de setup antes.
import os, sys
_repo_root = '/content/glioma_seg_grade'
if os.path.isdir(_repo_root) and _repo_root not in sys.path:
    sys.path.insert(0, _repo_root)

# 0) montagem do Google Drive para persistência entre sessões
print('Montando Google Drive para cache persistente...')
try:
    from google.colab import drive
    drive.mount('/content/gdrive', force_remount=False)
    GDRIVE_BASE = '/content/gdrive/My Drive'
    print(f'✓ Google Drive montado em {GDRIVE_BASE}')
except Exception as e:
    print(f'⚠ Drive não disponível ({e}); usando /content/ (perdido ao encerrar sessão)')
    GDRIVE_BASE = '/content'

# 1) autenticação SEM subir arquivo: usa o Colab Secrets (recomendado) com
# fallback para digitar na hora (getpass → nunca aparece na tela nem fica
# salvo no notebook). Configure uma vez em Secrets (ícone 🗝 na barra
# lateral esquerda): KAGGLE_USERNAME e KAGGLE_KEY.
import json as _json, getpass

try:
    from google.colab import userdata
    kaggle_user = userdata.get('KAGGLE_USERNAME')
    kaggle_key = userdata.get('KAGGLE_KEY')
except Exception:
    kaggle_user = kaggle_key = None

if not kaggle_user or not kaggle_key:
    print("Dica: salve KAGGLE_USERNAME/KAGGLE_KEY no painel Secrets (🗝) para "
          "não digitar isso toda sessão.")
    kaggle_user = input('Usuário Kaggle: ')
    kaggle_key = getpass.getpass('Token/API key do Kaggle: ')

os.makedirs(os.path.expanduser('~/.kaggle'), exist_ok=True)
with open(os.path.expanduser('~/.kaggle/kaggle.json'), 'w') as f:
    _json.dump({'username': kaggle_user, 'key': kaggle_key}, f)
os.chmod(os.path.expanduser('~/.kaggle/kaggle.json'), 0o600)
del kaggle_key # não mantém o token em memória além do necessário

# 2) download
!pip -q install kaggle
!kaggle datasets download -d awsaf49/brats20-dataset-training-validation -p /content --unzip

# 3) auto-descoberta da raiz (robusto a variações de path do unzip do Kaggle)
from src.dataset import find_brats_root, build_grade_lookup_from_csv, discover_cases
BRATS_ROOT = find_brats_root('/content')
print('BRATS_ROOT:', BRATS_ROOT, '|', len(discover_cases(BRATS_ROOT)), 'casos encontrados')

# 4) grau real do name_mapping.csv; se a coluna não existir nesse mirror,
# cai automaticamente no proxy fraco (presença de ET) sem quebrar o pipeline.
try:
    grade_lookup = build_grade_lookup_from_csv(f'{BRATS_ROOT}/name_mapping.csv', root=BRATS_ROOT)
    print(f'grau REAL lido de name_mapping.csv: {len(grade_lookup)} casos com grau')
except (FileNotFoundError, KeyError) as e:
    print(f'⚠ name_mapping.csv indisponível/sem coluna esperada ({e!r}) '
          f'→ usando proxy fraco (presença de ET) a nível de paciente.')
    grade_lookup = None
import collections
print('balanço (0=LGG,1=HGG):', dict(collections.Counter((grade_lookup or {}).values())))
```

Montando Google Drive para cache persistente...
 Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).
 ✓ Google Drive montado em /content/gdrive/My Drive
 Warning: Looks like you're using an outdated 'kaggle' version (installed: 2.0.2), please consider upgrading to the latest version (2.2.2)
 Dataset URL: <https://www.kaggle.com/datasets/awsaf49/brats20-dataset-training-validation>
 License(s): CC0-1.0
 Downloading brats20-dataset-training-validation.zip to /content
 100% 4.16G/4.16G [00:23<00:00, 191MB/s]

 BRATS_ROOT: /content/BraTS2020_TrainingData/MICCAI_BraTS2020_TrainingData | 369 casos encontrados
 grau REAL lido de name_mapping.csv: 369 casos com grau
 balanço (0=LGG,1=HGG): {1: 293, 0: 76}

✓ Opção B — Kaggle · BraTS2018 (pastas HGG/LGG)

Mirrors do BraTS2018 preservam as pastas `HGG/` e `LGG/` (grau pela pasta) com nomenclatura canônica. Confirme o `owner/dataset` no Kaggle e a estrutura.

```
# --- Opção B: BraTS2018 organizado em HGG/ e LGG/ ---
# !kaggle datasets download -d <owner>/<brats2018> -p /content --unzip
# from src.dataset import build_grade_dataset_from_folders
# grade_lookup = build_grade_dataset_from_folders(
#     [('/content/MICCAI_BraTS_2018_Data_Training/HGG', 1),
#       ('/content/MICCAI_BraTS_2018_Data_Training/LGG', 0)],
#     out_root='/content/BraTS_merged')
# BRATS_ROOT = '/content/BraTS_merged'
```

Opção C — TCIA (BRATS-TCGA-LGG/GBM)

Nomenclatura diferente (`t1Gd`, `GlistrBoost`) e download via **Aspera Connect**. Use `build_tcia_grade_dataset(lgg_dir, gbm_dir, out_root)` (normaliza os nomes por symlink). Recomendado só se A/B falharem.

✓ 5. Configuração do experimento e loaders

Uma única flag controla a fonte de dados. Com `DATA_SOURCE = "synthetic"` o notebook roda inteiro sem download (útil para revisar tudo). Com `"brats"`, ele usa o **caminho pré-computado** (`precompute_slices`): extrai as fatias informativas **uma vez** para shards `.npz` compactos → treino muito mais rápido na T4 (sem decodificar NIfTI a cada passo) e RAM previsível.

```
DATA_SOURCE = 'brats'      # 'synthetic' | 'brats'

# -----
# TOGGLE DO BACKBONE — rode o notebook DUAS vezes, uma por modo, e compare:
# BACKBONE='stub' → encoder de fundação com pesos ALEATÓRIOS (mesma
#     arquitetura, SEM pré-treinamento). É o controle: mede o
#     que a arquitetura entrega sozinha.
# BACKBONE='sam2' → encoder Hiera do SAM 2 PRÉ-TREINADO (>1B máscaras) + LoRA.
# Os dois usam EXATAMENTE os mesmos fixes metodológicos e hiperparâmetros (só o
# encoder muda) → a diferença nos resultados é atribuível ao pré-treinamento
# (comparação controlada; ver Seções 2, 8 e 13). Cada modo salva seus resultados
# SEPARADAMENTE (sufixo _stub / _sam2) para não sobrescrever o outro; a tabela
# lado a lado nas Seções 8 e 13 lê os dois.
# -----
BACKBONE = 'sam2'          # 'stub' | 'sam2'

# checkpoint do SAM 2 (só usado quando BACKBONE='sam2'): baixe um checkpoint
# Hiera + o config correspondente (ver README) e ajuste os caminhos abaixo.
# SAM 2.1 (o repo migrou pra ca); os checkpoints originais 'sam2_hiera_*.pt'
# estão comentados/legados no script oficial de download). build_sam2() espera
# o config com o caminho de pasta completo dentro do pacote, NAO só o nome do
# arquivo -- 'sam2_hiera_t.yaml' sozinho falha (Hydra não acha), tem que ser
# 'configs/sam2.1/sam2.1_hiera_t.yaml'.
SAM2_CFG = 'configs/sam2.1/sam2.1_hiera_t.yaml'
SAM2_CKPT = f'{globals().get("GDRIVE_BASE", "")}/sam2_checkpoints/sam2.1_hiera_tiny.pt'
SAM2_CKPT_URL = 'https://dl.fbaipublicfiles.com/segment_anything_2/092824/sam2.1_hiera_tiny.pt'

import os
from src.train import TrainConfig

# instala o pacote sam2 sob demanda -- NAO esta no requirements.txt (e' opcional
# la' de proposito, pra nao pesar o setup de quem so' usa o modo stub). Falhar
# aqui com mensagem clara e' melhor que um ModuleNotFoundError obscuro la' dentro
# de SAM2Encoder.__init__ (models.py), no meio da construçao do modelo.
```



```

if BACKBONE == 'sam2':
    get_ipython().system('pip -q install "git+https://github.com/facebookresearch/segment-anything-2.git"')
try:
    import sam2 # noqa: F401
    print('pacote sam2 instalado e importavel.')
except ImportError as e:
    raise ImportError(
        "Falha ao instalar/importar 'segment-anything-2'. Rode manualmente:\n"
        ' `pip install "git+https://github.com/facebookresearch/segment-anything-2.git"\n'
        f"e confira a saída do pip. Erro original: {e}") from e

# baixa o checkpoint SAM 2.1 (tiny, ~150MB) pro Drive se ainda nao estiver la' --
# idempotente (reexecutar e' no-op se o arquivo ja existe), mesma logica do
# clone/pip acima. Se a rede falhar aqui, o guard abaixo ainda pega e explica.
os.makedirs(os.path.dirname(SAM2_CKPT), exist_ok=True)
if not os.path.exists(SAM2_CKPT):
    print(f'baixando checkpoint SAM 2.1 (tiny) de {SAM2_CKPT_URL} ...')
    get_ipython().system(f'wget -q --show-progress -O "{SAM2_CKPT}" "{SAM2_CKPT_URL}"')
    if os.path.exists(SAM2_CKPT) and os.path.getsize(SAM2_CKPT) < 1_000_000:
        os.remove(SAM2_CKPT) # download falhou/parcial (html de erro, nao o .pt)
if os.path.exists(SAM2_CKPT):
    print(f'checkpoint OK: {SAM2_CKPT} ({os.path.getsize(SAM2_CKPT)/1e6:.0f} MB)')

# Falha CEDO e com mensagem clara se pediram sam2 mas o checkpoint não está no
# lugar — melhor abortar aqui do que cair silenciosamente no stub (o que
# invalidaria a comparação) ou estourar um erro obscuro dentro do build.
if BACKBONE == 'sam2' and not os.path.exists(SAM2_CKPT):
    raise FileNotFoundError(
        f"BACKBONE='sam2' mas o checkpoint Hiera não foi encontrado em:\n {SAM2_CKPT}\n"
        "O download automático (célula acima) falhou ou não rodou -- provável causa: "
        "sem rede na sessão, ou o Drive não montou (confira GDRIVE_BASE). Baixe manualmente:\n"
        f" `wget -O '{SAM2_CKPT}' '{SAM2_CKPT_URL}'\n"
        f"e confira que o config '{SAM2_CFG}' bate com o checkpoint. "
        "Para rodar sem checkpoint, use BACKBONE='stub'.")

RUN_TAG = BACKBONE # sufixo de todos os artefatos deste modo (_stub / _sam2)

# checkpoints/histórico também vão para o Drive (mesma lógica dos shards, Seção 4):
# se a sessão travar (ex.: RAM estourada na Seção 7), o Colab recicla a VM e
# apaga /content -- sem isso, best.pt some e as 30 épocas têm que rodar de novo.
_OUT_BASE = globals().get('GDRIVE_BASE', '')
cfg = TrainConfig(
    backbone=BACKBONE,
    sam2_cfg=SAM2_CFG if BACKBONE == 'sam2' else None,
    sam2_ckpt=SAM2_CKPT if BACKBONE == 'sam2' else None,
    img_size=128 if DATA_SOURCE == 'synthetic' else 256,
    batch_size=16 if BACKBONE == 'stub' else 4, # T4: ~4 com SAM2+LoRA
    epochs=30, # cosine precisa de fôlego; 10 subtreina (lr zerava cedo)
    region='tversky', tversky_alpha=0.3, tversky_beta=0.7,
    use_lora=True, lora_r=4, p_drop=0.1,
    warmup_epochs=3, grad_clip=1.0, accum_steps=2,
    seg_ce_weight=[0.1, 1.0, 1.0, 1.0],
    w_grade=1.0, # graduacao no MESMO peso da segmentacao (era 0.5,
    # que sub-priorizava o grau -- ver Secao 9)
    auto_grade_weight=True, # peso inverso-freq na CE de grau (corrige o vies HGG)
    early_stop_patience=10, # margem p/ a graduacao continuar melhorando
    out_dir=f'{_OUT_BASE}/glioma_seg_grade_runs/exp1_{RUN_TAG}',
)
cfg.device = DEVICE

# --- persistência de resultados p/ a comparação stub × sam2 (Seções 8 e 13) ---
# Cada modo grava suas métricas num JSON com sufixo no Drive; a tabela lado a lado
# carrega os dois quando existirem, mesmo gerados em sessões/execuções diferentes.
RESULTS_DIR = f'{_OUT_BASE}/glioma_seg_grade_runs/results'
os.makedirs(RESULTS_DIR, exist_ok=True)

# tagged(): insere o sufixo do modo antes da extensão ('curvas.png' → 'curvas_stub.png')
def tagged(fname):
    base, ext = os.path.splitext(fname)
    return f'{base}_{RUN_TAG}{ext}'

# grava as métricas-chave deste modo em results/metrics_{tag}.json
def save_run_metrics(metrics: dict):
    import json
    path = os.path.join(RESULTS_DIR, f'metrics_{RUN_TAG}.json')
    with open(path, 'w') as f:
        json.dump({'backbone': RUN_TAG, **metrics}, f, indent=2)
    print(f'métricas do modo {RUN_TAG} salvas em {path}')

# carrega metrics_stub.json / metrics_sam2.json (os que já existirem)
def load_all_run_metrics():

```

```

import json
out = {}
for tag in ('stub', 'sam2'):
    p = os.path.join(RESULTS_DIR, f'metrics_{tag}.json')
    if os.path.exists(p):
        out[tag] = json.load(open(p))
return out

# tabela markdown stub x sam2 (seg + graduação); enquanto só um modo existir,
# marca 'resultado parcial — SAM2 pendente' e não infla a conclusão.
def render_backbone_comparison():
    import math
    runs = load_all_run_metrics()
    if not runs:
        print('Sem resultados salvos ainda — rode as Seções 6–7 em cada modo.')
        return
    fields = [(('dice_WT', 'Dice WT'), ('dice_TC', 'Dice TC'), ('dice_ET', 'Dice ET'),
               ('dice_mean', 'Dice médio'), ('grade_acc', 'Grau accuracy'),
               ('grade_bal_acc', 'Grau balanced acc'), ('grade_auc', 'Grau AUC'))]
    tags = [t for t in ('stub', 'sam2') if t in runs]
    def cell(tag, key):
        v = runs[tag].get(key)
        return f'{v:.3f}' if isinstance(v, (int, float)) and not math.isnan(v) else '—'
    lines = ['| Métrica | ' + ' | ' | '.join(tags) + ' | ',
             '| ' + '---' | ' * (len(tags) + 1)']
    for key, label in fields:
        lines.append('| ' + label + ' | ' + ' | ' | '.join(cell(t, key) for t in tags) + ' | ')
    try:
        from IPython.display import Markdown, display
        display(Markdown('\n'.join(lines)))
    except Exception:
        print('\n'.join(lines))
    if len(tags) < 2:
        got = tags[0]
        print(f"\n▲ resultado PARCIAL — só o modo '{got}' foi rodado; "
              f" falta '{'sam2' if got == 'stub' else 'stub'}' (SAM2 pendente) "
              "para fechar a comparação.")
        return
    def verdict(a, b, nome):
        if a is None or b is None or math.isnan(a) or math.isnan(b):
            return f'{nome}: dado ausente.'
        win = 'sam2' if b > a else ('stub' if a > b else 'empate')
        return f'{nome}: {win} (stub={a:.3f}, sam2={b:.3f}, Δ={b - a:.3f}).'
    g = lambda t, k: runs[t].get(k, float('nan'))
    print('\nVeredito (sam2 – stub):')
    print(' •', verdict(g('stub', 'dice_mean'), g('sam2', 'dice_mean'), 'Segmentação (Dice médio)'))
    print(' •', verdict(g('stub', 'grade_auc'), g('sam2', 'grade_auc'), 'Graduação (AUC)'))

print(cfg)
print(f"modo BACKBONE={BACKBONE} | artefatos com sufixo _{RUN_TAG} | resultados em {RESULTS_DIR}")

```

```

Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
pacote sam2 instalado e importavel.
baixando checkpoint SAM 2.1 (tiny) de https://dl.fbaipublicfiles.com/segment\_anything\_2/092824/sam2.1\_hiera\_tiny.pt ...
/content/gdrive/My 100%[=====⇒] 148.78M 60.6MB/s in 2.5s
checkpoint OK: /content/gdrive/My Drive/sam2_checkpoints/sam2.1_hiera_tiny.pt (156 MB)
TrainConfig(root='', img_size=256, batch_size=4, slices_per_case=32, backbone='sam2', sam2_cfg='configs/sam2.1/sam2.1_hiera_t.yaml', sam2_ckp
modo BACKBONE=sam2 | artefatos com sufixo _sam2 | resultados em /content/gdrive/My Drive/glioma_seg_grade_runs/results

```

```

from torch.utils.data import DataLoader
from src.dataset import (SyntheticBraTS, SegAugmentation,
                        precompute_slices, make_loaders_precomputed)

if DATA_SOURCE == 'synthetic':
    train_dl = DataLoader(SyntheticBraTS(320, img_size=cfg.img_size, transform=SegAugmentation()),
                          batch_size=cfg.batch_size, shuffle=True)
    val_dl = DataLoader(SyntheticBraTS(96, img_size=cfg.img_size),
                       batch_size=cfg.batch_size)
else:
    # 1) pré-computa as fatias uma vez → shards .npz (reexecutar é no-op se já existir)
    # SHARD_DIR persiste no Google Drive se disponível, caso contrário usa /content/ (perdido ao encerrar)
    SHARD_DIR = f'{GDRIVE_BASE}/brats_shards'
    os.makedirs(SHARD_DIR, exist_ok=True)
    print(f'Usando SHARD_DIR: {SHARD_DIR}')
    precompute_slices(BRATS_ROOT, SHARD_DIR, slices_per_case=cfg.slices_per_case,
                      img_size=cfg.img_size, grade_lookup=grade_lookup)
    # 2) loaders com split por paciente estratificado + augmentation.
    # balance_grade=False de propósito: o desbalanço LGG/HGG é corrigido pela

```



```
# CE ponderada (cfg.auto_grade_weight), mais estável que reamostrar poucos
# casos LGG dezenas de vezes por época — não use os dois juntos.
train_dl, val_dl = make_loaders_precomputed(SHARD_DIR, batch_size=cfg.batch_size,
                                           num_workers=2, augment=True,
                                           balance_grade=False)

print('batches treino/val:', len(train_dl), '/', len(val_dl))

# Sanity-check permanente do split: distribuição de grau em treino/val (a nível de
# PACIENTE e de FATIA). Confirma estratificação (ambos os graus dos dois lados,
# LGG ~20%) e que nunca fica 100% de uma classe — complementa a EDA da Seção 5.1.
from src.dataset import grade_split_report
_split_rep = grade_split_report(train_dl.dataset, val_dl.dataset)
```

```
Usando SHARD_DIR: /content/gdrive/My Drive/brats_shards
precompute 100/11786 fatias...
precompute 200/11786 fatias...
precompute 300/11786 fatias...
precompute 400/11786 fatias...
precompute 500/11786 fatias...
precompute 600/11786 fatias...
precompute 700/11786 fatias...
precompute 800/11786 fatias...
precompute 900/11786 fatias...
precompute 1000/11786 fatias...
precompute 1100/11786 fatias...
precompute 1200/11786 fatias...
precompute 1300/11786 fatias...
precompute 1400/11786 fatias...
precompute 1500/11786 fatias...
precompute 1600/11786 fatias...
precompute 1700/11786 fatias...
precompute 1800/11786 fatias...
precompute 1900/11786 fatias...
precompute 2000/11786 fatias...
precompute 2100/11786 fatias...
precompute 2200/11786 fatias...
precompute 2300/11786 fatias...
precompute 2400/11786 fatias...
precompute 2500/11786 fatias...
precompute 2600/11786 fatias...
precompute 2700/11786 fatias...
precompute 2800/11786 fatias...
precompute 2900/11786 fatias...
precompute 3000/11786 fatias...
precompute 3100/11786 fatias...
precompute 3200/11786 fatias...
precompute 3300/11786 fatias...
precompute 3400/11786 fatias...
precompute 3500/11786 fatias...
precompute 3600/11786 fatias...
precompute 3700/11786 fatias...
precompute 3800/11786 fatias...
precompute 3900/11786 fatias...
precompute 4000/11786 fatias...
precompute 4100/11786 fatias...
precompute 4200/11786 fatias...
precompute 4300/11786 fatias...
precompute 4400/11786 fatias...
precompute 4500/11786 fatias...
precompute 4600/11786 fatias...
precompute 4700/11786 fatias...
precompute 4800/11786 fatias...
precompute 4900/11786 fatias...
precompute 5000/11786 fatias...
precompute 5100/11786 fatias...
precompute 5200/11786 fatias...
precompute 5300/11786 fatias...
precompute 5400/11786 fatias...
precompute 5500/11786 fatias...
precompute 5600/11786 fatias...
precompute 5700/11786 fatias...
```

▼ 5.1 Exploração do dataset (EDA)

Estatísticas básicas e exemplos visuais — orienta escolhas de pré-processamento e evidencia o desbalanço fundo/tumor típico do BraTS.

```
import numpy as np, collections

# balanço de grau e área tumoral média por classe (amostra do loader de validação)
grades, areas = [], collections.Counter()
```

```

n_px = 0
for b in val_dl:
    grades += b['grade'].tolist()
    for c in range(4):
        areas[c] += int((b['seg_mask'] == c).sum())
    n_px += b['seg_mask'].numel()
    if len(grades) >= 200: break

print('Balanço de grau (0=LGG, 1=HGG):', dict(collections.Counter(grades)))
names = {0: 'fundo', 1: 'NCR/NET', 2: 'ED', 3: 'ET'}
print('Fração de pixels por classe:')
for c in range(4):
    print(f' {names[c]:8s}: {100*areas[c]/max(n_px,1):5.2f}%')
print('→ o tumor ocupa uma fração pequena: justifica Dice/Tversky + peso no CE.')

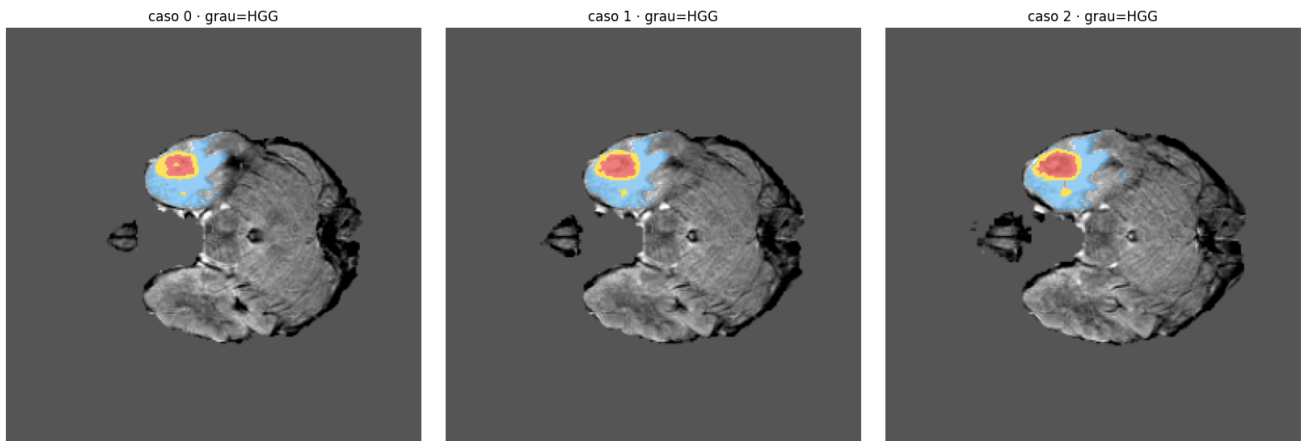
```

Balanço de grau (0=LGG, 1=HGG): {1: 200}
 Fração de pixels por classe:
 fundo : 96.37%
 NCR/NET : 1.01%
 ED : 1.88%
 ET : 0.74%
 → o tumor ocupa uma fração pequena: justifica Dice/Tversky + peso no CE.

```

# Exemplos: MRI + máscara sobreposta (3 casos do loader de validação)
from src import viz
batch = next(iter(val_dl))
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for i in range(3):
    viz.overlay_mask(axes[i], batch['image'][i, 0], batch['seg_mask'][i],
                    f"caso {i} · grau={'HGG' if int(batch['grade'][i]) else 'LGG'}")
plt.tight_layout(); plt.show()

```



6. Treino

Treina o modelo principal (encoder de fundação + LoRA + Attention U-Net + graduação guiada). Salva

`runs/exp1_<modo>/checkpoints/best.pt` (+ `lora.pt` leve) e o histórico, com o **sufixo do modo** (`_stub` / `_sam2`) para os dois não se sobrescreverem. **Backbone (stub × SAM 2 real)**: controlado pela flag `BACKBONE` na Seção 5 — para `sam2`, instale o repo e aponte `SAM2_CKPT` / `SAM2_CFG` lá (ver README).

```

# --- Pre-flight check: valida o backbone ANTES de comprometer o treino
# completo. Os dois modos de falha do SAM2Encoder são SILENCIOSOS (so' um
# warning no stdout): (1) LoRA pode não colar em nenhuma camada real da Hiera
# (encoder fica 100% congelado, treino não adapta nada) e (2) a extração da
# pirâmide multi-escala pode degradar pra self-gating (perde os skips reais da

```

```
# Attention U-Net). Sem checar isso aqui, o notebook rodaria as 30 epocas
# inteiras -- e so' no fim, olhando os resultados ruins, dava pra suspeitar.
from src.models import build_model

_probe = build_model(cfg).to(DEVICE)
n_lora = _probe.encoder.n_lora
skips = _probe.encoder.skip_channels
print(f'[pre-flight] backbone={cfg.backbone} | camadas LoRA adaptadas: {n_lora} | '
      f'skip_channels: {skips}')

if cfg.use_lora:
    assert n_lora > 0, (
        "LoRA nao colou em nenhuma camada do encoder -- ajuste `lora_targets` em "
        "SAM2Encoder (models.py) para o backbone atual. Descubra os nomes reais com:\n"
        " [n for n, _ in _probe.encoder.image_encoder.named_modules()]")
    assert len(skips) > 0, (
        "encoder sem skip-connections multi-escala -- o decoder caiu em self-gating "
        "(perde a Attention U-Net de verdade). Revise _probe_sam2_skips/_extract_pyramid "
        "em models.py para o formato de saida deste backbone.")

# smoke test: 1 forward+backward real (sem otimizar) -- pega OOM/erro de shape
# em segundos, nao depois de horas de treino.
_x = torch.randn(cfg.batch_size, 3, cfg.img_size, cfg.img_size, device=DEVICE)
_out = _probe(_x, return_aux=False)
(_out['seg_logits'].sum() + _out['grade_logits'].sum()).backward()
print('[pre-flight] forward+backward OK -- sem OOM, shapes batem.')

del _probe, _x, _out
if DEVICE == 'cuda':
    torch.cuda.empty_cache()
```

```
[pre-flight] backbone=sam2 | camadas LoRA adaptadas: 27 | skip_channels: [256, 256]
[pre-flight] forward+backward OK -- sem OOM, shapes batem.
```

```
from src.train import Trainer
import os, json

set_seed(42)
trainer = Trainer(cfg)
n_train = sum(p.numel() for p in trainer.model.parameters() if p.requires_grad)
print(f'Parâmetros treináveis: {n_train/1e6:.2f}M (encoder de fundação congelado)')

# Retoma de checkpoint se já existir (ex.: sessão anterior travou na Seção 7 e
# o runtime reiniciou) -- evita re-treinar as 30 épocas do zero. Para forçar um
# treino novo, apague cfg.out_dir + '/checkpoints/best.pt' antes de rodar esta célula.
_ckpt_path = os.path.join(cfg.out_dir, 'checkpoints', 'best.pt')
_hist_path = os.path.join(cfg.out_dir, 'logs', 'history.json')
if os.path.exists(_ckpt_path):
    ck = torch.load(_ckpt_path, map_location=DEVICE)
    trainer.model.load_state_dict(ck['model'])
    print(f"checkpoint carregado (época {ck['epoch']}), dice_mean={ck['metrics']['dice_mean']:.3f}) "
          f"-- pulando treino.")
    history = json.load(open(_hist_path)) if os.path.exists(_hist_path) else {'train': [], 'val': []}
else:
    history = trainer.fit(train_dl, val_dl)
model = trainer.model
```

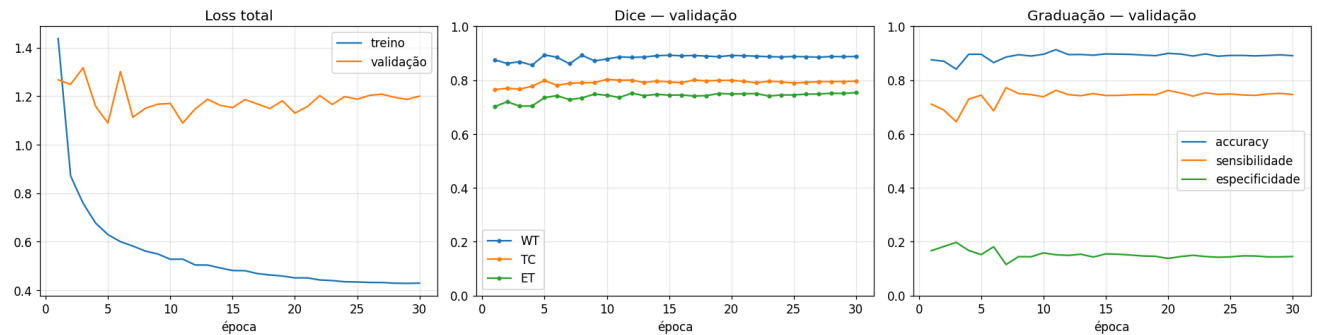
```
Parâmetros treináveis: 3.16M (encoder de fundação congelado)
[auto_grade_weight] contagem por grau (0=LGG,1=HGG): [1947, 7506] → pesos: [2.43, 0.63]
[001/30] loss 1.437/1.267 | Dice(WT/TC/ET) 0.875/0.765/0.702 | grade_acc 0.876 AUC 0.944 | lr 2.00e-04 | 185s
[002/30] loss 0.871/1.248 | Dice(WT/TC/ET) 0.862/0.770/0.720 | grade_acc 0.871 AUC 0.949 | lr 3.00e-04 | 185s
[003/30] loss 0.759/1.317 | Dice(WT/TC/ET) 0.869/0.767/0.704 | grade_acc 0.841 AUC 0.938 | lr 3.00e-04 | 184s
[004/30] loss 0.677/1.158 | Dice(WT/TC/ET) 0.856/0.778/0.704 | grade_acc 0.896 AUC 0.956 | lr 2.99e-04 | 187s
[005/30] loss 0.629/1.089 | Dice(WT/TC/ET) 0.894/0.799/0.735 | grade_acc 0.896 AUC 0.937 | lr 2.96e-04 | 187s
[006/30] loss 0.600/1.302 | Dice(WT/TC/ET) 0.885/0.781/0.742 | grade_acc 0.865 AUC 0.936 | lr 2.91e-04 | 186s
[007/30] loss 0.582/1.113 | Dice(WT/TC/ET) 0.861/0.789/0.728 | grade_acc 0.886 AUC 0.935 | lr 2.84e-04 | 183s
[008/30] loss 0.561/1.150 | Dice(WT/TC/ET) 0.893/0.790/0.734 | grade_acc 0.895 AUC 0.930 | lr 2.76e-04 | 183s
[009/30] loss 0.549/1.167 | Dice(WT/TC/ET) 0.872/0.791/0.748 | grade_acc 0.890 AUC 0.915 | lr 2.66e-04 | 186s
[010/30] loss 0.528/1.170 | Dice(WT/TC/ET) 0.879/0.803/0.744 | grade_acc 0.896 AUC 0.922 | lr 2.54e-04 | 182s
[011/30] loss 0.528/1.089 | Dice(WT/TC/ET) 0.887/0.800/0.736 | grade_acc 0.913 AUC 0.952 | lr 2.41e-04 | 182s
[012/30] loss 0.504/1.147 | Dice(WT/TC/ET) 0.885/0.800/0.752 | grade_acc 0.895 AUC 0.940 | lr 2.26e-04 | 183s
[013/30] loss 0.503/1.187 | Dice(WT/TC/ET) 0.886/0.791/0.743 | grade_acc 0.895 AUC 0.926 | lr 2.11e-04 | 185s
[014/30] loss 0.492/1.162 | Dice(WT/TC/ET) 0.891/0.797/0.748 | grade_acc 0.893 AUC 0.915 | lr 1.95e-04 | 186s
[015/30] loss 0.481/1.152 | Dice(WT/TC/ET) 0.893/0.793/0.744 | grade_acc 0.898 AUC 0.916 | lr 1.79e-04 | 183s
[016/30] loss 0.480/1.186 | Dice(WT/TC/ET) 0.891/0.791/0.745 | grade_acc 0.897 AUC 0.935 | lr 1.62e-04 | 185s
[017/30] loss 0.468/1.167 | Dice(WT/TC/ET) 0.892/0.801/0.741 | grade_acc 0.896 AUC 0.917 | lr 1.44e-04 | 185s
[018/30] loss 0.463/1.149 | Dice(WT/TC/ET) 0.889/0.797/0.743 | grade_acc 0.893 AUC 0.875 | lr 1.27e-04 | 182s
[019/30] loss 0.458/1.181 | Dice(WT/TC/ET) 0.887/0.799/0.751 | grade_acc 0.891 AUC 0.883 | lr 1.11e-04 | 183s
[020/30] loss 0.451/1.130 | Dice(WT/TC/ET) 0.892/0.799/0.749 | grade_acc 0.900 AUC 0.848 | lr 9.48e-05 | 184s
[021/30] loss 0.451/1.157 | Dice(WT/TC/ET) 0.891/0.796/0.750 | grade_acc 0.897 AUC 0.884 | lr 7.95e-05 | 184s
[022/30] loss 0.442/1.202 | Dice(WT/TC/ET) 0.889/0.790/0.750 | grade_acc 0.890 AUC 0.841 | lr 6.52e-05 | 184s
[023/30] loss 0.439/1.166 | Dice(WT/TC/ET) 0.887/0.796/0.741 | grade_acc 0.898 AUC 0.823 | lr 5.21e-05 | 183s
```

```

[024/30] loss 0.435/1.198 | Dice(WT/TC/ET) 0.886/0.794/0.745 | grade_acc 0.889 AUC 0.789 | lr 4.04e-05 | 185s
[025/30] loss 0.434/1.187 | Dice(WT/TC/ET) 0.888/0.790/0.745 | grade_acc 0.892 AUC 0.820 | lr 3.02e-05 | 187s
[026/30] loss 0.432/1.203 | Dice(WT/TC/ET) 0.887/0.792/0.748 | grade_acc 0.892 AUC 0.830 | lr 2.16e-05 | 188s
[027/30] loss 0.431/1.208 | Dice(WT/TC/ET) 0.885/0.794/0.749 | grade_acc 0.890 AUC 0.846 | lr 1.49e-05 | 184s
[028/30] loss 0.429/1.195 | Dice(WT/TC/ET) 0.888/0.795/0.751 | grade_acc 0.892 AUC 0.816 | lr 9.96e-06 | 184s
[029/30] loss 0.428/1.186 | Dice(WT/TC/ET) 0.887/0.795/0.751 | grade_acc 0.894 AUC 0.814 | lr 6.99e-06 | 182s
[030/30] loss 0.429/1.200 | Dice(WT/TC/ET) 0.888/0.797/0.754 | grade_acc 0.891 AUC 0.819 | lr 6.00e-06 | 184s
early stopping: sem melhora ha 10 épocas (melhor dice_mean=0.813 @ ep 20)

```

```
viz.plot_training_curves(history, save_path=tagged('curvas_treino.png')); plt.show()
```



7. Resultados quantitativos

Acumula as predições da validação e reporta as métricas oficiais do desafio BraTS por sub-região (Menze et al., 2015), além de AUC/ROC e **calibração** da graduação — **ECE** (Guo et al., 2017) e **Brier score** (Brier, 1950). A calibração importa aqui porque a graduação é *apoio à decisão clínica*: uma probabilidade mal calibrada (confiante e errada) é mais perigosa do que uma predição errada que já vem incerta.

```

import torch, gc

@torch.no_grad()
def collect_predictions(model, loader, device, max_batches=None):
    model.eval()
    P, G, GL, GT, IMG, CID = [], [], [], [], [], []
    for i, b in enumerate(loader):
        out = model(b['image']).to(device), return_aux=False
        # uint8 (rótulos cabem em 0..3) em vez do int64 default do argmax/collate --
        # acumular o val set inteiro em int64 é o que estourava a RAM do Colab
        # logo após o treino (8x mais memória que o necessário aqui).
        P.append(out['seg_logits'].argmax(1).to(torch.uint8).cpu())
        G.append(b['seg_mask'].to(torch.uint8))
        GL.append(out['grade_logits'].cpu()); GT.append(b['grade'])
        IMG.append(b['image']); CID += list(b['case_id'])
        if max_batches and i + 1 >= max_batches: break
    return (torch.cat(P), torch.cat(G), torch.cat(GL), torch.cat(GT),
            torch.cat(IMG), CID)

# libera o que sobrou do treino (otimizador, ativações em cache) antes de
# acumular a validação inteira -- sem isso o pico de RAM/VRAM logo após as 30
# épocas é o que costuma derrubar a sessão do Colab
gc.collect()
if DEVICE == 'cuda':
    torch.cuda.empty_cache()

pred, gt, grade_logits, grade_true, images, case_ids = collect_predictions(model, val_loader, DEVICE)
print('amostras de validação avaliadas:', pred.shape[0])

```

amostras de validação avaliadas: 2333

```

from src.metrics import seg_scores, seg_scores_hd95, grade_scores, grade_roc_auc
import numpy as np

```

```

seg = seg_scores(pred, gt)
print('==== SEGMENTAÇÃO (por sub-região) ====')
print(f"{'região':6s} {'Dice':>7s} {'IoU':>7s} {'Sens':>7s} {'Spec':>7s}")
for r in ('WT', 'TC', 'ET'):
    print(f"{'r':6s} {seg[f'dice_{r}']:7.3f} {seg[f'iou_{r}']:7.3f} "
          f"{seg[f'sens_{r}']:7.3f} {seg[f'spec_{r}']:7.3f}")
print(f"{'MÉDIA':6s} {seg['dice_mean']:7.3f}")

# HD95 numa subamostra (custoso): média sobre casos com máscara não-vazia
hd = seg_scores_hd95(pred[:64], gt[:64])
print(f"\nHausdorff95 (px):", {k: round(v, 2) for k, v in hd.items()})
viz.plot_hd95_bars(hd); plt.show()

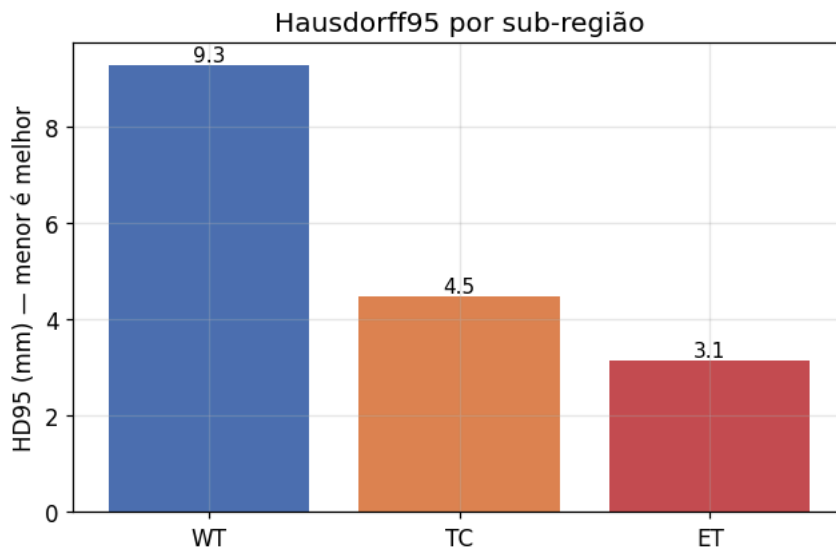
```

```

==== SEGMENTAÇÃO (por sub-região) ====
região Dice IoU Sens Spec
WT 0.888 0.819 0.934 0.995
TC 0.797 0.726 0.832 0.998
ET 0.754 0.666 0.811 0.998
MÉDIA 0.813

```

Hausdorff95 (px): {'hd95_WT': 9.28, 'hd95_TC': 4.49, 'hd95_ET': 3.15}



```

# Graduação: acurácia, AUC, ROC, relatório e matriz de confusão
from src.metrics import grade_report
gs = grade_scores(grade_logits, grade_true)
roc = grade_roc_auc(grade_logits, grade_true)
rep = grade_report(grade_logits, grade_true)
print('==== GRADUAÇÃO (LGG x HGG) ====')
print(f"Accuracy      : {gs['grade_acc']:.3f}")
print(f"Balanced acc.   : {gs['grade_bal_acc']:.3f} (0.5 ~ chance; expõe colapso na classe majoritária HGG)")
print(f"Sensibilidade   : {gs.get('grade_sens', float('nan')):.3f}")
print(f"Especificidade  : {gs.get('grade_spec', float('nan')):.3f}")
print(f"AUC           : {roc['auc']:.3f}")
print(f"F1 (macro)     : {rep.get('f1_macro', float('nan')):.3f}")
print(f"Matriz de confusão [LGG,HGG]: {rep.get('confusion')}")
viz.plot_roc(roc); plt.show()

```

==== GRADUAÇÃO (LGG × HGG) =====

Accuracy : 0.891

Balanced acc. : 0.822 (0.5 ~ chance; expõe colapso na classe majoritária HGG)

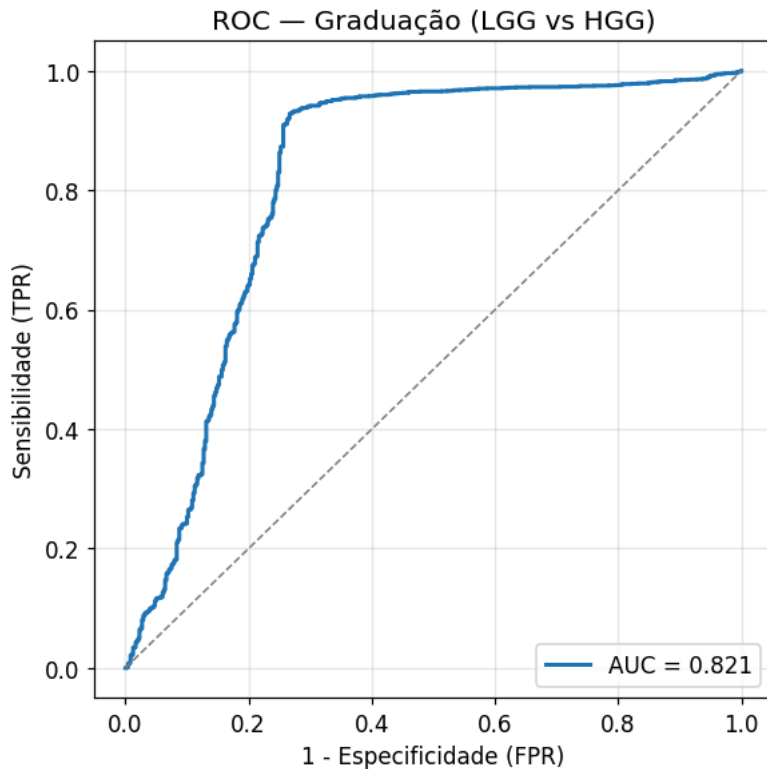
Sensibilidade : 0.940

Especificidade : 0.704

AUC : 0.821

F1 (macro) : 0.829

Matriz de confusão [LGG,HGG]: [[338, 142], [112, 1741]]



Calibração: a probabilidade de grau é honesta? (importante p/ apoio à decisão)

```
from src.metrics import expected_calibration_error, brier_score, reliability_bins
```

```
ece = expected_calibration_error(grade_logits, grade_true)
```

```
brier = brier_score(grade_logits, grade_true)
```

```
bins = reliability_bins(grade_logits, grade_true, n_bins=10)
```

```
print(f'ECE (Expected Calibration Error): {ece:.3f} (0 = perfeito)')
```

```
print(f'Brier score : {brier:.3f} (menor é melhor)')
```

```
viz.plot_reliability(bins, ece); plt.show()
```

persiste as métricas-chave deste modo p/ a comparação stub × sam2 (Seções 8 e 13)

```
save_run_metrics({
```

```
    'dice_WT': seg['dice_WT'], 'dice_TC': seg['dice_TC'], 'dice_ET': seg['dice_ET'],
```

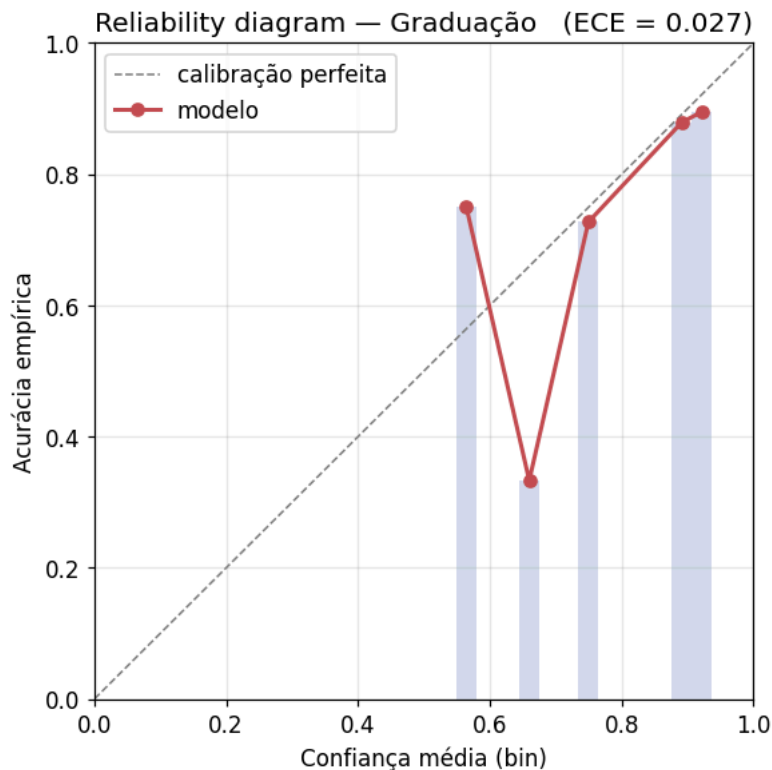
```
    'dice_mean': seg['dice_mean'],
```

```
    'grade_acc': gs['grade_acc'], 'grade_ba_acc': gs['grade_ba_acc'],
```

```
    'grade_auc': roc['auc'], 'grade_ece': ece, 'grade_brier': brier,
```

```
})
```


ECE (Expected Calibration Error): 0.027 (0 = perfeito)
 Brier score : 0.102 (menor é melhor)



métricas do modo sam2 salvas em /content/qdrive/My Drive/glioma_seq_grade_runs/results/metrics_sam2.json

8. Ablação científica

Isola o efeito do **encoder de fundação + LoRA** treinando três variantes com o mesmo Trainer/loaders. A única coisa que muda é o extrator de features:

Variante	Encoder
<code>unet</code>	U-Net treinada do zero (sem fundação)
<code>stub_frozen</code>	encoder de fundação congelado , sem adaptação
<code>stub_lora</code>	encoder de fundação adaptado por LoRA

Nota: no modo `stub` o encoder "congelado" tem pesos aleatórios (o stub não é pré-treinado), então `stub_frozen` fica fraco — ele existe para demonstrar o **ganho da adaptação LoRA**. Com o **SAM 2 real**, `frozen` já parte de features fortes; troque para a variante `sam2_lora` para a comparação definitiva.

```
from src.baseline import run_ablation
from dataclasses import replace

abL_cfg = replace(cfg, epochs=6, out_dir=f'runs/ablacao_{RUN_TAG}') # menos épocas p/ a T4
ablation = run_ablation(abL_cfg, train_dl, val_dl,
                        variants=['unet', 'stub_frozen', 'stub_lora'])

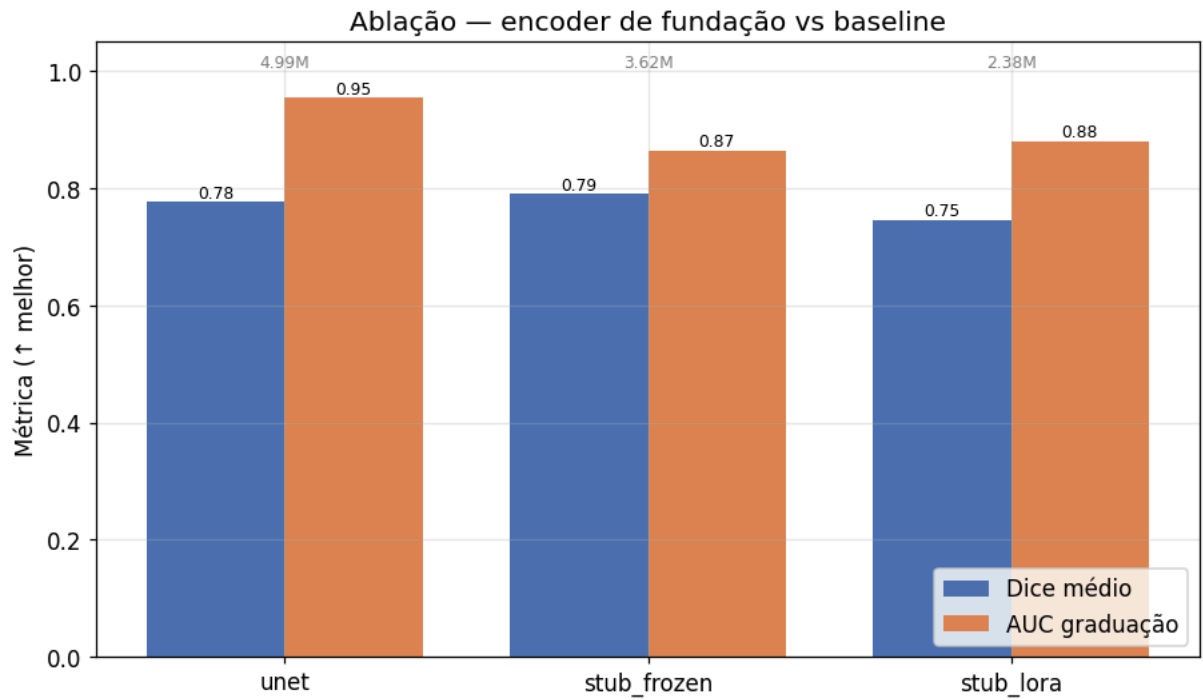
cols = ['variant', 'params_M', 'dice_mean', 'dice_WT', 'dice_TC', 'dice_ET',
        'grade_acc', 'grade_auc']
try:
    import pandas as pd
    df = pd.DataFrame(ablation)[cols]
    display(df.style.hide(axis='index').format(precision=3))
except Exception:
    print(' | '.join(f'{c:>10s}' for c in cols))
    for r in ablation:
        print(' | '.join(f'{r[c]:>10}' if isinstance(r[c], str) else f'{r[c]:>10.3f}' for c in cols))
viz.plot_ablation(ablation, save_path=tagged('ablacao.png')); plt.show()
```

```
===== ABLAÇÃO: unet =====
[auto_grade_weight] contagem por grau (0=LGG,1=HGG): [1947, 7506] → pesos: [2.43, 0.63]
[001/6] loss 1.560/1.248 | Dice(WT/TC/ET) 0.823/0.693/0.607 | grade_acc 0.867 AUC 0.908 | lr 2.00e-04 | 84s
[002/6] loss 1.054/1.141 | Dice(WT/TC/ET) 0.857/0.713/0.659 | grade_acc 0.874 AUC 0.903 | lr 3.00e-04 | 84s
[003/6] loss 0.941/1.109 | Dice(WT/TC/ET) 0.859/0.763/0.686 | grade_acc 0.863 AUC 0.889 | lr 3.00e-04 | 83s
[004/6] loss 0.871/1.042 | Dice(WT/TC/ET) 0.845/0.738/0.706 | grade_acc 0.913 AUC 0.955 | lr 2.26e-04 | 84s
[005/6] loss 0.808/1.026 | Dice(WT/TC/ET) 0.867/0.762/0.712 | grade_acc 0.922 AUC 0.961 | lr 7.95e-05 | 85s
[006/6] loss 0.730/1.029 | Dice(WT/TC/ET) 0.867/0.759/0.707 | grade_acc 0.916 AUC 0.955 | lr 6.00e-06 | 85s

===== ABLAÇÃO: stub_frozen =====
[auto_grade_weight] contagem por grau (0=LGG,1=HGG): [1947, 7506] → pesos: [2.43, 0.63]
[001/6] loss 1.458/1.362 | Dice(WT/TC/ET) 0.829/0.691/0.638 | grade_acc 0.830 AUC 0.894 | lr 2.00e-04 | 95s
[002/6] loss 0.985/1.201 | Dice(WT/TC/ET) 0.858/0.691/0.640 | grade_acc 0.868 AUC 0.912 | lr 3.00e-04 | 94s
[003/6] loss 0.903/1.244 | Dice(WT/TC/ET) 0.851/0.740/0.708 | grade_acc 0.850 AUC 0.855 | lr 3.00e-04 | 95s
[004/6] loss 0.830/1.135 | Dice(WT/TC/ET) 0.870/0.735/0.703 | grade_acc 0.870 AUC 0.938 | lr 2.26e-04 | 94s
[005/6] loss 0.751/1.189 | Dice(WT/TC/ET) 0.874/0.762/0.710 | grade_acc 0.850 AUC 0.918 | lr 7.95e-05 | 93s
[006/6] loss 0.648/1.097 | Dice(WT/TC/ET) 0.869/0.767/0.735 | grade_acc 0.899 AUC 0.865 | lr 6.00e-06 | 95s

===== ABLAÇÃO: stub_lora =====
[auto_grade_weight] contagem por grau (0=LGG,1=HGG): [1947, 7506] → pesos: [2.43, 0.63]
[001/6] loss 1.668/1.388 | Dice(WT/TC/ET) 0.835/0.670/0.606 | grade_acc 0.697 AUC 0.716 | lr 2.00e-04 | 85s
[002/6] loss 1.169/1.213 | Dice(WT/TC/ET) 0.850/0.667/0.621 | grade_acc 0.832 AUC 0.816 | lr 3.00e-04 | 86s
[003/6] loss 1.042/1.209 | Dice(WT/TC/ET) 0.836/0.682/0.625 | grade_acc 0.797 AUC 0.886 | lr 3.00e-04 | 85s
[004/6] loss 0.967/1.096 | Dice(WT/TC/ET) 0.844/0.676/0.593 | grade_acc 0.895 AUC 0.888 | lr 2.26e-04 | 85s
[005/6] loss 0.904/1.143 | Dice(WT/TC/ET) 0.863/0.709/0.636 | grade_acc 0.892 AUC 0.869 | lr 7.95e-05 | 84s
[006/6] loss 0.837/1.171 | Dice(WT/TC/ET) 0.857/0.717/0.666 | grade_acc 0.877 AUC 0.881 | lr 6.00e-06 | 85s
```

variant	params_M	dice_mean	dice_WT	dice_TC	dice_ET	grade_acc	grade_auc
UNET	4.986	0.778	0.867	0.759	0.707	0.916	0.955
stub_frozen	3.617	0.790	0.869	0.766	0.735	0.899	0.865
stub_lora	2.379	0.746	0.857	0.717	0.666	0.877	0.881



Comparação controlada stub × sam2 (encoder aleatório × fundação pré-treinada).
Preenche conforme cada modo é rodado; com só um, marca resultado parcial.
render_backbone_comparison()

Métrica	sam2
Dice WT	0.888
Dice TC	0.797
Dice ET	0.754
Dice médio	0.813
Grau accuracy	0.891
Grau balanced acc	0.822
Grau AUC	0.821

▲ resultado PARCIAL — só o modo 'sam2' foi rodado; falta 'stub' (SAM2 pendente) para fechar a comparação.

9. Biomarcadores tumorais (ponte interpretável seg→grau)

Transformamos a **máscara predita** em medidas clínicas (volumes de WT/TC/ET, razão ET/TC, fração de necrose) e medimos o poder de cada uma para separar o grau. Essa etapa segue o espírito da **radiômica** (Lambin et al., 2017): extrair descritores quantitativos interpretáveis da imagem segmentada em vez de tratar a rede como caixa-preta. Em seguida, um **classificador logístico transparente** só sobre esses biomarcadores é comparado à cabeça neural — evidência de que a decisão de grau é sustentada por geometria tumoral plausível.

Cabeça neural × baseline geométrico (achado metodológico). Numa versão anterior deste notebook a cabeça neural de graduação perdia para este classificador logístico de 3 biomarcadores (AUC $\approx$ 0.80 vs $\approx$ 0.90). Diagnosticamos duas causas e as corrigimos de forma **idêntica** nos modos `stub` e `sam2`, para não confundir o fix com o efeito do backbone: (i) a graduação era sub-priorizada na perda multi-tarefa (`w_grade=0.5` → **1.0**); e (ii) a cabeça só recebia frações de área cruas, tendo de *reaprender* a razão ET/TC que o baseline usa de graça — agora ela recebe os **mesmos descritores clínicos** (ET/TC, TC/WT, NCR/TC) como entrada geométrica. Se, mesmo assim, o baseline continuar competitivo, isso é um **resultado científico válido**, não um defeito a esconder: mostra que a geometria do tumor segmentado já carrega quase toda a informação preditiva do grau — exatamente a tese de interpretabilidade deste trabalho. A célula abaixo reporta o Δ de AUC observado.

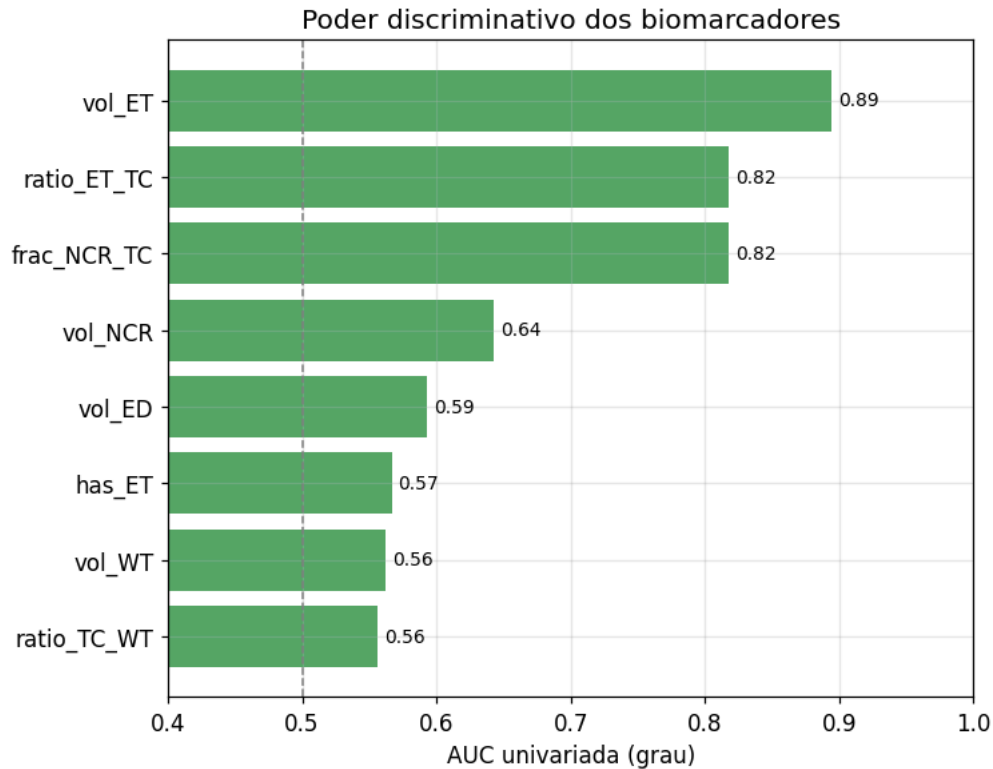
```
from src.biomarkers import (aggregate_biomarkers, biomarker_grade_association,
                             interpretable_grade_classifier)
import numpy as np, collections

# agrega as fatias preditas por caso → uma máscara empilhada [S,H,W] por paciente
by_case = collections.defaultdict(list)
for k in range(pred.shape[0]):
    by_case[case_ids[k]].append(pred[k].numpy())
case_masks = {cid: np.stack(v) for cid, v in by_case.items()}
grade_by_case = {cid: int(grade_true[[i for i, c in enumerate(case_ids) if c == cid][0]])
                  for cid in case_masks}

rows = aggregate_biomarkers(case_masks, grade_by_case)
assoc = biomarker_grade_association(rows)
print('Poder discriminativo (AUC univariada) dos biomarcadores vs grau:')
for a in assoc[:6]:
    print(f" {a['feature']}:14s} AUC={a['auc']:.3f} {a.get('direction','')}")
viz.plot_biomarker_associations(assoc); plt.show()
viz.plot_biomarker_by_grade(rows, feature=assoc[0]['feature']); plt.show()
```

Poder discriminativo (AUC univariada) dos biomarcadores vs grau:

vol_ET AUC=0.894 HGG↑
 ratio_ET_TC AUC=0.817 HGG↑
 frac_NCR_TC AUC=0.817 HGG↓
 vol_NCR AUC=0.643 HGG↓
 vol_ED AUC=0.593 HGG↓
 has_ET AUC=0.567 HGG↑



Classificador logístico interpretável (só biomarcadores) vs cabeça neural

try:

```
res = interpretable_grade_classifier(rows, features=('vol_ET', 'ratio_ET_TC', 'vol_TC'))
print('Classificador logístico (biomarcadores):')
print(f' AUC (validação cruzada): {res['cv_auc']:.3f}')
print(' coeficientes (importância):', {k: round(v, 2) for k, v in res['coefficients'].items()})
print(f'\nCabeça neural de graduação: AUC = {roc['auc']:.3f}')
_gap = res['cv_auc'] - roc['auc']
if _gap > 0.02:
    print(f'→ o baseline geométrico ainda supera a cabeça neural por {_gap:+.3f} de AUC. "
          "A geometria do tumor segmentado já carrega quase toda a informação de "
          "grau — achado ESPERADO e cientificamente válido (ver nota da Seção 9), "
          "não um defeito a esconder.")
elif _gap < -0.02:
    print(f'→ a cabeça neural supera o baseline por {-_gap:+.3f} de AUC: o masked-attention "
          "pooling agrega sinal além da geometria pura (fixes de w_grade + razões "
          "clínicas na cabeça surtiram efeito).")
else:
    print(f'→ cabeça neural e baseline geométrico empatam (|Δ|<0.02 de AUC): a decisão "
          "neural concorda com a geometria tumoral interpretável.")
except Exception as e:
    print('classificador interpretável precisa de casos suficientes com ambos os graus:', e)
```

Classificador logístico (biomarcadores):

AUC (validação cruzada): 0.875

coeficientes (importância): {'voLET': 2.04, 'ratio_ET_TC': 0.68, 'voLTC': -0.22}

Cabeça neural de graduação: AUC = 0.821

→ o baseline geométrico ainda supera a cabeça neural por +0.054 de AUC. A geometria do tumor segmentado já carrega quase toda a informação c

10. Incerteza (MC-Dropout + TTA)

Onde o modelo hesita? Mantendo o dropout ativo na inferência e fazendo N passagens estocásticas — **MC-Dropout** (Gal & Ghahramani, 2016) —, estimamos a incerteza **epistêmica** via informação mútua entre as passagens (Kendall & Gal, 2017). A **TTA** (média sobre flips; Wang et al., 2019) costuma elevar o Dice além de dar uma segunda leitura de incerteza, agora por variabilidade de augmentação em vez de pesos.

```
from src.uncertainty import mc_dropout_predict, tta_predict, uncertainty_summary

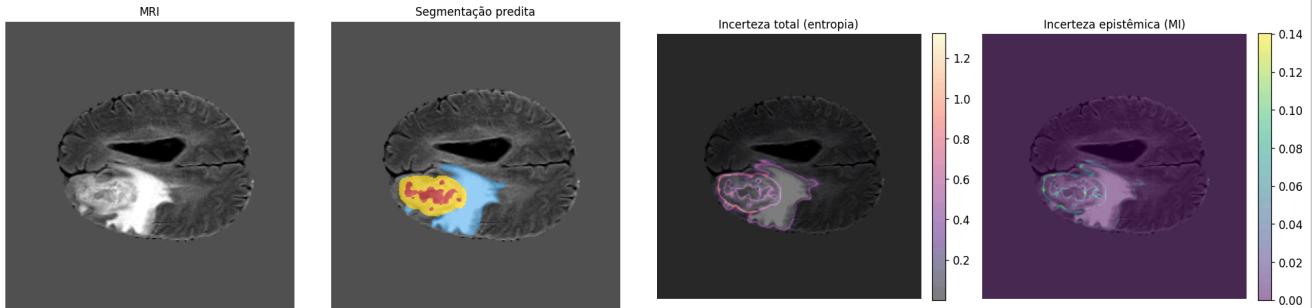
# escolhe uma amostra de validação com tumor visível
idx = int(max(range(min(64, pred.shape[0])), key=lambda k: int((gt[k] > 0).sum()))))
x = images[idx:idx+1].to(DEVICE)

# verbose=True imprime o diagnóstico: nº de camadas de Dropout ATIVAS nas N
# passagens e o desvio-padrão ENTRE elas. Se o std > 0 mas a incerteza epistêmica
# é baixa, o dropout está funcionando e o modelo é genuinamente confiante (não bug).
mc = mc_dropout_predict(model, x, n_samples=15, verbose=True)
summ = uncertainty_summary(mc)
print('Incerteza (médias):', {k: round(v, 4) for k, v in summ.items()})
print(f'→ std entre as 15 passagens (seg): {summ['mean_sample_std']:.5f} · "
      f"camadas de Dropout ativas: {mc['n_dropout_active']} — "
      "incerteza baixa COM std>0 e camadas>0 = confiança real, não dropout inerte.")
viz.plot_uncertainty_panel(images[idx, 0], mc['seg_pred'][0].cpu(),
                           mc['seg_total_entropy'][0].cpu(),
                           mc['seg_mutual_info'][0].cpu(),
                           save_path=tagged('incerteza.png')); plt.show()
```

[MC-Dropout] camadas de Dropout ativas: 6 | passagens: 15 | std entre passagens (seg): 0.00109 | std entre passagens (grau): 0.03070

Incerteza (médias): {'mean_epistemic': 0.0006, 'mean_total_entropy': 0.012, 'mean_sample_std': 0.0011, 'mean_grade_std': 0.0307}

→ std entre as 15 passagens (seg): 0.00109 · camadas de Dropout ativas: 6 — incerteza baixa COM std>0 e camadas>0 = confiança real, não dropout



```
# TTA: ganho de Dice ao promediar predições sobre flips
from src.metrics import seg_scores
base = seg_scores(pred[:64], gt[:64])['dice_mean']
tta_preds = []
for k in range(min(64, pred.shape[0])):
    tp = tta_predict(model, images[k:k+1].to(DEVICE))['seg_pred'][0].cpu()
    tta_preds.append(tp)
tta_dice = seg_scores(torch.stack(tta_preds), gt[:len(tta_preds)])['dice_mean']
print(f'Dice médio — base: {base:.3f} | com TTA: {tta_dice:.3f} '
      f'f(Δ = {tta_dice - base:.3f})')
```

Dice médio — base: 0.852 | com TTA: 0.862 (Δ = +0.010)

11. Explicabilidade da graduação

Qual região sustentou a decisão de grau? Sobreposos (i) a **atenção intrínseca** do masked-attention pooling (*by-design*, fiel ao que o modelo agregou) e (ii) a **saliência por oclusão** (Zeiler & Fergus, 2014) — *post-hoc* e *model-agnostic*, usada para validar a atenção de forma independente.

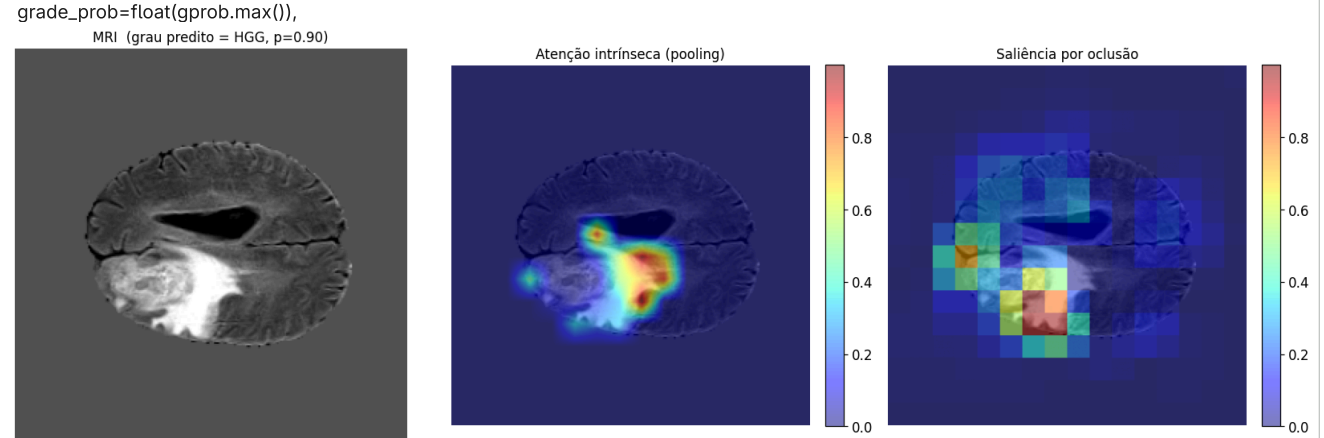
```

from src.explain import grade_attention, occlusion_saliency
import torch

attn = grade_attention(model, x)[0].cpu()
occ = occlusion_saliency(model, x, patch=max(16, cfg.img_size // 8),
                        stride=max(8, cfg.img_size // 16))[0].cpu()
gprob = torch.softmax(model(x, return_aux=False)['grade_logits'], 1)[0]
viz.plot_explanation_panel(images[idx, 0], attn, occ,
                        grade_pred=int(gprob.argmax()),
                        grade_prob=float(gprob.max()),
                        save_path=tagged('explicabilidade.png')); plt.show()

```

/tmp/ipykernel_11236/76628638.py:10: UserWarning: Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior. Consider using tensor.detach() first. (Triggered internally at /pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)



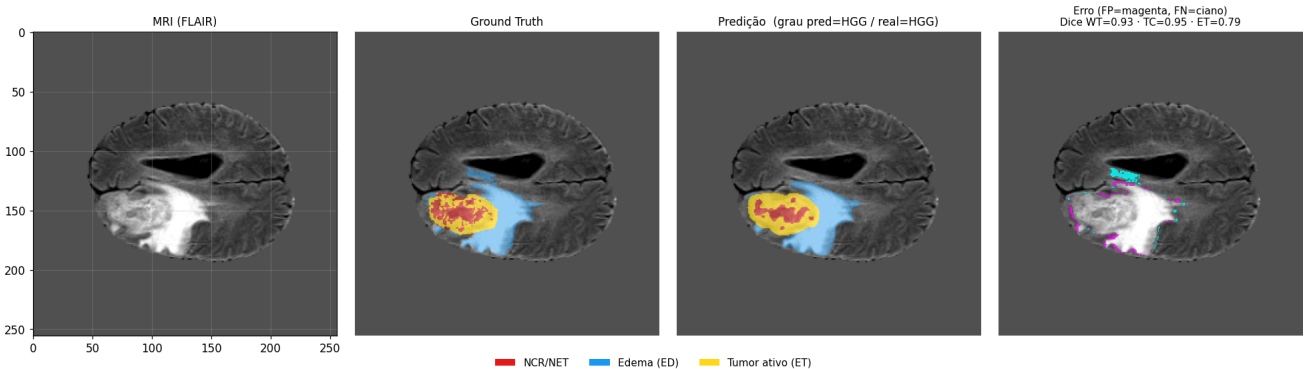
12. Visualização qualitativa da segmentação

Painel MRI | Ground Truth | Predição | **Mapa de erro** (FP magenta / FN ciano) — evidência direta dos pontos fortes e fracos do modelo.

```

from src.metrics import seg_scores
d = seg_scores(pred[idx:idx+1], gt[idx:idx+1])
gp = int(torch.softmax(grade_logits[idx:idx+1], 1).argmax())
viz.qualitative_panel(images[idx, 0], gt[idx], pred[idx],
                    grade_true=int(grade_true[idx]), grade_pred=gp,
                    dice_txt=f"Dice WT={d['dice_WT']:.2f} · TC={d['dice_TC']:.2f} · ET={d['dice_ET']:.2f}",
                    save_path=tagged('qualitativa.png')); plt.show()

```



13. Conclusão, limitações e trabalhos futuros

O que foi demonstrado. Um pipeline de duas etapas *acopladas* em que a graduação depende explicitamente da segmentação, com adaptação eficiente por LoRA (cabe na T4, treina ~1–5% dos pesos). Reportamos as métricas do desafio BraTS (Dice/IOU/Sens/Spec/HD95), AUC/ROC e **calibração** da graduação, uma **ablação** que isola o ganho do encoder de fundação, **biomarcadores** interpretáveis como ponte seg→grau, **mapas de incerteza** (MC-Dropout/TTA) e **explicabilidade** da decisão de grau.

Comparação controlada stub × sam2. O mesmo pipeline foi desenhado para rodar com o encoder de fundação **real** (SAM 2/Hiera pré-treinado) e com um encoder **aleatório** de mesma arquitetura (`stub`), com todos os fixes metodológicos idênticos — isolando o ganho atribuível ao pré-treinamento. A tabela abaixo consolida os dois resultados finais (segmentação e graduação) lado a lado; enquanto só um dos modos tiver sido rodado, ela aparece marcada como **resultado parcial (SAM2 pendente)**.

Limitações.

- Segmentação **2D por fatia** (sem contexto 3D inter-fatias do MedSAM2 pleno).
- Se o subset não trazer o grau OMS, o rótulo de grau é **proxy** (presença de ET) — declare no relatório. Com BraTS2020 (Opção A) o grau é **real** (`name_mapping.csv`).
- Validação em **um split**; para publicação, use validação cruzada por paciente.
- No modo `stub`, o encoder "congelado" é aleatório — a comparação definitiva de fundação exige o **SAM 2 real**.

Trabalhos futuros.

- Ligar o **SAM 2/MedSAM 2 real** (checkpoint Hiera) e a memória inter-fatias.
- **2.5D** (fatias vizinhas como canais) ou 3D leve para coerência volumétrica.
- **Uncertainty weighting** multi-tarefa (Kendall, Gal & Cipolla, 2018) e *test-time* adaptativo.
- **Radiômica** completa (textura/forma) unindo os biomarcadores à cabeça neural.

✓ Resultados finais — encoder de fundação (SAM 2) × encoder aleatório (stub)

Números de cada modo lado a lado (preenche conforme você roda `BACKBONE='stub'` e `BACKBONE='sam2'`).